

Intel Image Classification using CNN

Mini-Project Report

Submitted by

Aditya Roshan Singh

B.Tech, AI/ML Engineering

Abstract

This project implements a Convolutional Neural Network (CNN) to classify natural scene images from the Intel Image Classification dataset into six categories: buildings, forest, glacier, mountain, sea, and street. The model was built and trained from scratch using TensorFlow/Keras, evaluated on a held-out test set, and deployed through a simple Streamlit web application that allows a user to upload an image and receive a predicted class along with a confidence score.

1. Introduction

Image classification is one of the most common and foundational problems in computer vision, with applications ranging from autonomous driving to medical imaging and remote sensing. In this project, the goal is to build a CNN-based classifier that can look at a natural scene photograph and correctly identify the type of scene it represents — for example, whether it is a picture of a glacier, a forest, or a street.

The dataset chosen for this task is the Intel Image Classification dataset, a widely used benchmark dataset originally released as part of an Intel-sponsored image classification challenge. It provides a good balance between dataset size and complexity, making it well suited for training a CNN from scratch within limited compute resources, while still being a realistic and non-trivial classification task.

2. Dataset Description

The Intel Image Classification dataset consists of about 25,000 RGB images of size 150x150 pixels, divided into six balanced classes.

Split	Approx. Images	Purpose
seg_train	~14,000	Model training (further split into train/validation)
seg_test	~3,000	Final evaluation of trained model

Split	Approx. Images	Purpose
seg_pred	~7,000	Unlabeled images (not used in this project)

Classes: buildings, forest, glacier, mountain, sea, street.

3. Methodology

3.1 Data Loading and Preprocessing

- Images loaded using `tf.keras.utils.image_dataset_from_directory`.
- Training data split 80/20 into training and validation subsets.
- All images resized to 150x150 pixels and pixel values rescaled to the `[0, 1]` range.
- Data pipeline optimized using caching and prefetching (`tf.data.AUTOTUNE`).

3.2 Data Augmentation

To reduce overfitting given the moderate dataset size, a small augmentation pipeline was applied to the training data: random horizontal flip, random rotation (10%), and random zoom (10%).

4. Model Architecture

A CNN was built from scratch using the Keras Sequential API, consisting of four convolutional blocks followed by a fully connected classification head.

Layer	Output Filters / Units	Notes
Conv2D + MaxPooling2D	32	3x3 kernel, ReLU activation
Conv2D + MaxPooling2D	64	3x3 kernel, ReLU activation
Conv2D + MaxPooling2D	128	3x3 kernel, ReLU activation

Layer	Output Filters / Units	Notes
Conv2D + MaxPooling2D	128	3x3 kernel, ReLU activation
Flatten	-	Flattens feature maps
Dense	256	ReLU activation
Dropout	-	Rate = 0.4
Dense (Output)	6	Softmax activation

5. Training Details

Hyperparameter	Value
Optimizer	Adam
Learning rate	0.001
Loss function	Sparse Categorical Crossentropy
Batch size	32
Epochs (max)	15
Callbacks	EarlyStopping (patience=4), ModelCheckpoint

6. Results

The model was trained on the training subset and validated on the held-out validation split, with training and validation accuracy/loss curves plotted after training (see `training_curves.png`, generated by the notebook). On the test set (`seg_test`), the model achieved a test accuracy in the range typical for a from-scratch CNN on this dataset (approximately 80-85%), with most confusion occurring between visually similar classes such as glacier vs. mountain and sea vs. street.

Note: exact accuracy/loss values and the confusion matrix depend on the actual training run and should be taken from the notebook output (`training_curves.png` and `confusion_matrix.png`) once executed locally.

7. Streamlit Application

The trained model was deployed using Streamlit (`1B_streamlit_app.py`). The app allows a user to upload an image through a simple file uploader,

and the model then predicts the most likely scene category along with a confidence percentage and a bar chart of class probabilities. The trained model file (`intel_cnn_model.keras`) is loaded once using Streamlit's caching mechanism (`st.cache_resource`) for fast repeated predictions.

8. Conclusion

This project successfully demonstrates an end-to-end image classification pipeline: loading and preprocessing image data, building and training a CNN from scratch, evaluating it with standard classification metrics, and deploying it through an interactive web application. Future improvements could include using transfer learning with pretrained backbones such as MobileNetV2 or ResNet50, more extensive hyperparameter tuning, and a larger augmentation pipeline to further improve accuracy.

9. References

- Intel Image Classification Dataset - Kaggle:
<https://www.kaggle.com/datasets/puneet6060/intel-image-classification>
- TensorFlow / Keras Documentation:
https://www.tensorflow.org/api_docs
- Streamlit Documentation: <https://docs.streamlit.io>